

SVEUČILIŠTE U ZAGREBU
FAKULTET STROJARSTVA I BRODOGRADNJE

ZAVRŠNI RAD

Petra Alimović

Zagreb, godina 2026.

SVEUČILIŠTE U ZAGREBU
FAKULTET STROJARSTVA I BRODOGRADNJE

ZAVRŠNI RAD

Mentori:

Prof. dr. sc. Marko Švaco, dipl. ing.

Student:

Petra Alimović

Zagreb, godina 2026.

Izjavljujem da sam ovaj rad izradio samostalno koristeći znanja stečena tijekom studija i navedenu literaturu.

Zahvaljujem se mentoru prof. dr. sc. Marku Švaci što mi je pružio priliku za izradu ovog rada, te na njegovom vodstvu, uvidima i savjetima.

Zahvaljujem se laboratoriju CRTA i svim njegovim zaposlenicima na pomoći i opremi potrebnoj za izradu ovog rada.

Zahvaljujem se svojoj obitelji na dugogodišnjoj podršci tijekom pisanja ovog rada i tijekom cijelog studija.

Naposljetku, posebno se zahvaljujem svim prijateljima i kolegama koji su mi pomogli svojim idejama, savjetima i podrškom. Bez njih, ni početak niti završetak ovog rada ne bi bio moguć.

Petra Alimović

[Ovdje umetnuti "Završni zadatak"]

SADRŽAJ

SADRŽAJ	I
POPIS SLIKA	III
POPIS TABLICA	IV
POPIS OZNAKA	V
SAŽETAK	VI
SUMMARY	VII
1. UVOD	1
2. MOBILNI ROBOTI	2
2.1. ASTRO robot	3
2.2. Arhitektura ASTRO robota	3
3. SENZORSKI SUSTAV	4
3.1. Enkoderi	4
3.2. Intel RealSense D435 Depth kamera	4
3.3. LiDAR senzor	5
4. ROS2 SUSTAV	6
5. MEDIAPIPE BIBLIOTEKA	8
6. OPEN CV BIBLIOTEKA	10
7. SUSTAV ZA UPRAVLJANJE POMOĆU GESTI RUKE	12
7.1. Prepoznavanje geste ruke	12
7.2. Izračun brzina robota	17
7.3. Simulacija	23
7.4. Integracija s mobilnim robotom ASTRO	26

8. SUSTAV ZA ZAOKLAŽENJE PREPREKA	28
9. ZAKLJUČAK	29
LITERATURA	29
PRILOZI	31

POPIS SLIKA

1	Slika	2
---	-----------------	---

POPIS TABLICA

POPIS OZNAKA

Oznaka	Jedinica	Opis
Oznaka 1	Jedinica	Opis oznake
Oznaka 2	Jedinica	Opis oznake
Oznaka 3	Jedinica	Opis oznake
Oznaka 4	Jedinica	Opis oznake

SAŽETAK

Tekst sažetka.

Ključne riječi:

SUMMARY

Abstract.

Key words:

1. UVOD

Mobilni roboti su automatizirani, programabilni, multifunkcionalni mehatronički sustavi sposobni kretati se autonomno, to jest bez pomoći ljudskih operatera, kroz prostor kako bi izvršavali zadane zadatke. [citati iz literature] To znači da robot sam ima sposobnost određivanja postupaka potrebnih za izvršavanje zadatka, koristeći senzore i algoritme za percepciju okoline. [citati iz literature]

Kada se ne kreću u potpunosti autonomno, njima je potrebno upravljati koristeći upravljačke palice, tipkovnicu, miša ili sličnu opremu. U novije vrijeme, mobilni roboti su sve pristupačniji te je njihova primjena sve šira i uključuje područja poput industrije, medicine, vojske, poljoprivrede, ali i u kućanstvima te za osobnu uporabu. To stvara potrebu za razvojem novih metoda upravljanja koja će omogućiti jednostavnije i intuitivnije upravljanje mobilnim robotima uz korištenje što manje opreme. Iz tog razloga, razvijeni su načini upravljanja koji se oslanjaju na glasovne naredbe, pokrete glave i očiju te geste ruku i tijela. Upravljanje mobilnim robotima pomoću gesti ruku posebno je zanimljivo jer omogućuje korisniku da upravlja robotom bez potrebe za složenom opremom, a pritom zadržava visoku razinu preciznosti.

Problem kod upravljanja ASTRO robotom je što ne postoji ugrađeni sustav za zaštitu od sudara pri nailasku na prepreku. Ukoliko korisnik, ili sam robot, napravi krivi pokret, krivo procijeni udaljenost ili brzinu kretanja, nema dovoljno dobru povratnu informaciju o okolini i slično, može doći do zabijanja u prepreke, oštećenja robota, ozljeda korisnika ili druge štete. Kako bi se to izbjeglo, potrebno je razviti sustav za zaobilazanje prepreka. Tehnike za izbjegavanje prepreka baziraju se na detekciji prepreka u prostoru i planiranju putanje kretanja koristeći podatke dobivene sa senzora robota kako bi se izbjegao sudar. Implementacijom ovog sustava, mobilni robot dobiva veći stupanj autonomije. Ovakav sustav primjenjiv je na sve robote, pa tako i mobilni robot ASTRO, koji imaju implementiran LiDAR senzor, neovisno o njihovom stupnju autonomije. Sustav djeluje kao zaštitni sloj (engl. safety layer) visoke razine prioriteta koji prema potrebi preuzima kontrolu nad kretanjem robota, zanemarujući ručne naredbe operatera dok se ne otkloni opasnost od sudara.

Cilj ovog završnog rada jest razviti i implementirati sustave za upravljanje mobilnim robotom u stvarnom vremenu koristeći upravljačke naredbe dobivene gestama ruke te za detekciju, automatsko zaobilazanje prepreka i izbjegavanje sudara.

2. MOBILNI ROBOTI

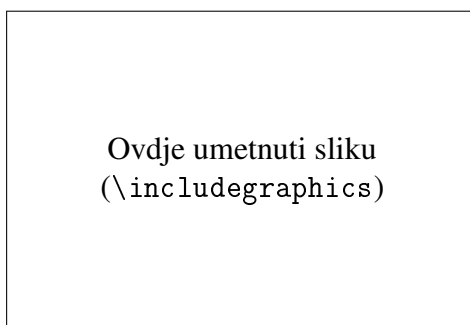
Mobilni roboti se od ostalih razlikuju svojom sposobnošću kretanja po prostoru koja im je omogućena integracijom pogonskih, senzorskih i upravljačkih sustava. Svaki od tih sustava međusobnom integracijom omogućuje pravilan rad robota, a nalazi se u konstrukciji robota koja ga štiti od vanjskih utjecaja.

Pogonski sustav odgovoran je za svako gibanje robota. Aktuatori koji se najčešće koriste su elektromotori istosmjerne struje, sa ili bez četkica. Na njih su većinom ugrađeni enkoderi kojima se mjeri kut zakreta motora, a time i brzina, akceleracija te procjena pozicije.

Glavna funkcija senzorskog sustava je prikupljanje informacija iz okoline koje obrađuje upravljački sustav. Senzori su od velike važnosti jer omogućuju robotu da bude svjestan svoje okoline i promjena u njoj, što mu je potrebno za autonomno donošenje odluka. Najčešće korišteni senzori su LiDAR, kamere, ultrazvučni senzori, infracrveni senzori, IMU (engl. Inertial Measurement Unit) i GPS (engl. Global Positioning System).

Upravljački sustav povezuje dva prethodno navedena sustava. Njegova osnova je mikrokontroler koji obrađuje podatke dobivene sa senzora i na temelju njih donosi odluke te upravlja aktuatorima kako bi ih proveo. Ovisno o složenosti robota, upravljački sustav može biti implementiran na mikrokontroleru, računalu ili na više njih. U slučaju kada se koristi više mikrokontrolera ili računala, sustav dijelimo na upravljanje visoke razine (engl. high-level control) i upravljanje niske razine (engl. low-level control). Najčešće korišteni prograski jezici za programiranje upravljačkih sustava su C++ i Python.

Glavna podjela mobilnih robota vrši se na temelju medija u kojem se koriste, tlo, zrak ili voda. Podvodni i zračni roboti kreću se u tri dimenzije, dok su kopneni ograničeni na planarno kretanje. Zbog široke primjenjivosti, kopneni mobilni roboti su najrašireniji i najčešće korišteni, pa tako i u ovom radu.



Slika 1. Slika

2.1. ASTRO robot

ASTRO robot (Autonomus System for Teaching RObotics) je mobilni robot razvijen u regionalnom centru izvrsnosti za robotske tehnologije CRTA za potrebe edukacije i istraživačkog razvoja. Za svrhu ovog rada, ASTRO nudi robusnu platformu jednostavnu za korištenje na koju su implementirani sustav za upravljanje gestama ruke i sustav za izbjegavanje prepreka.

2.2. Arhitektura ASTRO robota

Nadređeno upravljačko računalo na robotu je jednopločno računalo Nvidia Jetson Nano s operacijskim sustavom Ubuntu Linux i ROS 2 (Robot Operating System 2) Humble, sposobno za cijelokupnu obradu podataka. Jetson Nano je malo računalo napravljeno za učenje, razvoj alata i proizvoda te drugih primjena u granama umjetne inteligencije i robotike.

Upravljanje niske razine u stvarnom vremenu obavlja se putem mikrokontrolera Teensy 4.0, izgrađenog oko procesora ARM Cortex-M7 i spojenog na Jetson Nano putem sučelja UART (Universal Asynchronous Receiver Transmitter). Teensy pretvara zadane linearne i kutne brzine lijevog i desnog kotača te zatvara PiD pretlju regulacije brzine na temelju povratne informacije s enkodera.

Cijela arhitektura ASTRO robota prikazana je na Slika [broj].

Za pokretanje ROS 2 čvorova koji omogućuju serijsku komunikaciju između komponenti koristi se Docker. Logika nadređenog sustava implementirana je u ROS 2 prostoru, a pomoću pokrenutih čvorova primaju se informacije sa senzora, upravljačkih palica ili informacije koje šalju vanjski spojeni uređaji.

Prikaz cijele ROS 2 strukture vidi se na Slika [broj].

3. SENZORSKI SUSTAV

ASTRO koristi sljedeće senzore:

1. enkoderi integrirani u pogonske motore
2. Intel RealSense D435 Depth kamera
3. Slamtec RPLIDAR A1 LiDAR.

Svaki od ovih senzora je korišten tijekom izrade ovog rada.

Osim senzora integriranih na ASTRO-u, korištena je još jedna Intel RealSense D435 Depth kamera postavljena izvan robota i spojena na vanjsko računalo koje je povezano s robotom i upravlja njime.

3.1. Enkoderi

Enkoderi mjere rotaciju svakog pogonskog kotača, iz čega se izračunava i akumulira prijeđeni put radi određivanja odometrije robota. Svaki motor opremljen je magnetskim enkoderom koji daje 16 impulsa po okretaju uz reduktor s prijenosnim omjerom 43,8:1, a prijeđeni putevi pojedinih kotača kombiniraju se pomoću modela diferencijalnog pogona kako ni se izračunala pozicija robota. Enkoderi pružaju visoku brzinu osvježavanja te su jednostavni za integraciju. U ovom radu, odometrija s enkodera koristila se pri simulaciji kretanja robota putem razvijenog sustava za upravljanjem pomoću gesti ruke.

3.2. Intel RealSense D435 Depth kamera

Intel RealSense D435 širokokutna je stereo kamera za percepciju dubine, namijenjena primjenama u robotici i računalnom vidu. Kombinira par infracrvenih stereo kamera s aktivnim infracrvenim projektorom i RGB kamerom u boji, što joj omogućuje generiranje gustih, metričkih mapa dubine pri brzini do 90 slika u sekundi i doseg od otprilike 0,2 do 10 metara. Dubina se izračunava na samom uređaju pomoću namjenskog čipa za stereokorekciju, koji na scenu projicira strukturirani infracrveni uzorak kako bi poboljšao teksturu i preciznost procjene dubine u područjima bez izražene teksture. Dobiveni trodimenzionalni oblak točaka, zajedno s poravnatom slikom u boji, dostupan je spojenom računalu putem Intel RealSense SDK 2.0 i njegovih Python biblioteka (pyrealsense2), koje su u ovom radu korištene za prikupljanje sinkroniziranih okvira boje i dubine. Kamera se na računalo spaja putem USB 3.0 sučelja i ne zahtijeva vanjsko napajanje, što je čini praktičnim i prenosivim senzorom za mobilne robotske platforme. U ovom radu korištene su dvije ovakve kamere. Prva kamera nalazila se na ASTRO robotu te služila za povratnu informaciju o poziciji robota u prostoru koja je dostupna operateru unutar korisničkog

sučelja. Druga kamera bila je postavljena izvan robota i korištena za snimanje operaterove ruke u stvarnom vremenu, pri čemu je okvir dubine pružao metričku trodimenzionalnu poziciju ruke u odnosu na kameru, koja je potom korištena za izračun upravljačkih naredbi za robota.

3.3. LiDAR senzor

Slamtec RPLIDAR A1 je dvodimenzionalni laserski daljinomjer koji je sposoban izvesti potpuni sken okoline u rasponu od 360 stupnjeva. Radi na principu optičke triangulacije, tj. emitira infracrvenu lasersku zraku i mjeri kut pod kojim se reflektirana svjetlost vraća na pozicijski osjetljivi detektor, iz čega se izračunava udaljenost do reflektirajuće površine. Senzor se neprekidno rotira brzinom od otprilike 5,5 okretaja u sekundi, prikupljajući do 8000 mjerenja udaljenosti u sekundi pri kutnoj rezoluciji od približno jednog stupnja, s radnim rasponom od 0,15 do 12 metara. Sirovi podaci skena prenose se putem UART sučelja i dostupni su u ROS 2 okruženju kao standardna poruka tipa `sensor_msgs/LaserScan` posredstvom službenog upravljačkog paketa `slidar_ros2` koji pruža Slamtec, generirajući temu koju je izravno koristio čvor za izbjegavanje prepreka razvijen u ovom radu. Dobiveni sken pruža planarni presjek okoline na visini montaže senzora, što je dovoljno za detekciju prepreka i mjerenje slobodnih udaljenosti u horizontalnoj ravnini. U ovom radu RPLIDAR A1 bio je montiran na gornju stranu robota ASTRO i korišten kao primarni izvor za detekciju prepreka, omogućujući algoritmu za izbjegavanje praćenje okoline robota u svim smjerovima i određivanje sigurnih putanja kretanja.

4. ROS2 SUSTAV

ROS 2 (Robot Operating System 2) skup je alata i biblioteka otvorenog koda namijenjen izgradnji i integraciji aplikacija za robote. Unatoč svom nazivu, ROS 2 nije operativni sustav u tradicionalnom smislu, već skup alata, biblioteka i komunikacijskog međusloja (engl. middleware) koji neovisnim softverskim modulima, zvanim čvorovi (engl. nodes), omogućuje razmjenu podataka na strukturiran i standardiziran način. Svaki čvor je samostalan izvršni proces odgovoran za jedan, jasno definirani zadatak. Primjerice, u ovom su radu to su namjenski čvorovi zaduženi za snimanje gesta ruke, generiranje upravljačkih naredbi brzine, izbjegavanje prepreka te multipleksiranje izvora brzine.

Primarni mehanizam komunikacije u ROS 2 su teme (engl. topics), koje implementiraju asinkroni model objavi-pretplati (engl. publish-subscribe). Čvor koji proizvodi podatke objavljuje poruke na imenovanom kanalu teme, dok se bilo koji broj drugih čvorova može pretplatiti na taj isti kanal i primiti te poruke bez ikakve izravne sprege između objavitelja i pretplatnika. Ovakva odvojena arhitektura sustav čini inherentno modularnim: pojedini čvorovi mogu se razvijati, testirati, mijenjati ili iznova pokretati neovisno, bez narušavanja ostatka sustava.

Svi podaci koji se razmjenjuju putem tema prenose se u porukama (engl. messages). One su strogo tipizirane podatkovne strukture definirane jezikom za definiranje sučelja ROS-a (IDL, engl. Interface Definition Language). Vrsta poruke definira polja i njihove tipove podataka, osiguravajući da svi čvorovi koji dijele neku temu budu suglasni oko formata razmijenjenih podataka. Primjeri poruka korištenih u ovom radu uključuju `geometry_msgs/Twist` za upravljačke naredbe brzine, `sensor_msgs/LaserScan` za podatke o dometima LiDAR senzora te prilagođenu vrstu poruke `HandPose` koja nosi trodimenzionalnu poziciju operaterove ruke zajedno s prepoznatom gestom i ocjenom pouzdanosti.

Uz teme, ROS 2 pruža i servise (engl. services), koji implementiraju sinhroni komunikacijski obrazac zahtjev-odgovor: jedan čvor djeluje kao poslužitelj i izlaže imenovanu servisnu krajnju točku, dok klijentski čvor šalje tipizirani zahtjev i čeka tipizirani odgovor prije nastavka izvođenja. Servisi su prikladni za rijetke, jednokratne operacije poput upita o konfiguraciji ili promjene načina rada, za razliku od kontinuiranih tokova podataka za koje su teme namijenjene.

U ovom radu upravljanje naredbama gibanja robota preuzima paket `twist_mux`, koji se pretplaćuje na nekoliko konkurentnih tema tipa `geometry_msgs/Twist` koje generiraju različiti čvorovi: čvor za upravljanje joystickom, čvor za upravljanje gestama ruke i čvor za izbjegavanje prepreka. Paket prosljeđuje samo poruku izvora s najvišim konfiguriranim prioritetom upravljaču robota. Svaki izvor ima pridruženo konfigurirano vremensko ograničenje. Ako izvor prestane objavljivati unutar tog vremenskog okvira, automatski gubi prednost i upravljanje prelazi na sljedeći dostupni

izvor, čime se osigurava sigurno ponašanje u slučaju kvara čvora ili prekida komunikacije.

Verzija ROS 2 koja se koristi u ovom radu je Humble Hawksbill, objavljena u svibnju 2022. godine s dugotrajnom podrškom zajamčenom do svibnja 2027. godine.

5. MEDIAPIPE BIBLIOTEKA

MediaPipe je međuplatformski okvir otvorenog koda koji je razvio Google za izgradnju cjevovoda percepcije (engl. perception pipeline) u stvarnom vremenu temeljenih na strojnom učenju. Umjesto jedne namjenske aplikacije, nudi biblioteku unaprijed istreniranih modela i modularnih komponenti za obradu zasnovanih na grafovima, koje se mogu međusobno povezivati radi formiranja potpunih rješenja za zadatke poput detekcije lica, procjene položaja tijela, detekcije objekata i praćenja ruke. Ključni cilj dizajna MediaPipea je učinkovitost. Njegovi su modeli optimizirani za rad u stvarnom vremenu na standardnom procesorskom sklopovlju bez potrebe za grafičkim procesorom, što ga čini praktičnim za primjenu na ugradbenim i mobilnim platformama kakve su uobičajene u robotici.

Komponenta korištena u ovom radu je MediaPipe Hands, rješenje za praćenje ruke koje radi kroz dvostupanjski cjevovod strojnog učenja. U prvom stupnju, lagani model za detekciju dlana pronalazi ruku na ulaznoj slici i vraća područje omeđivanja. U drugom stupnju, zasebni model za detekciju ključnih točaka ruke uzima izrezanu i normaliziranu sliku detektiranog područja te regresijom određuje položaje 21 trodimenzionalne ključne točke koje opisuju skeletnu strukturu ruke. Te su ključne točke postavljene na anatomske definirane položaje: zapešće, metakarpofalangealni (MCP) zglobovi kostura dlana, proksimalni interfalangealni (PIP) zglobovi, distalni interfalangealni (DIP) zglobovi i vrhovi prstiju za svaki od četiri prsta, zajedno s četiri odgovarajuće točke duž palca. Koordinate x i y svake ključne točke izražene su u normaliziranom prostoru slike u rasponu od nule do jedan u odnosu na dimenzije slike, dok koordinata z kodira relativnu dubinu unutar ruke, pri čemu zapešće služi kao referentno ishodište.

Nakon što je ruka jednom detektirana, cjevovod prelazi iz načina detekcije u način praćenja za sljedeće sličice, koristeći prethodne položaje ključnih točaka za vođenje učinkovitije operacije izrezivanja i poravnavanja, umjesto ponovnog pokretanja punog detektora dlana u svakom kadru. Detekcija se nastavlja tek kada ruka napusti kadar ili kada pouzdanost modela padne ispod konfigurabilnog minimalnog praga. Klasifikator lateralnosti dodatno dodjeljuje oznaku lijeve ili desne ruke detektiranoj ruci zajedno s ocjenom pouzdanosti. U ovom su radu oba praga pouzdanosti, za detekciju i za praćenje, postavljena na 0,6, a sličice ispod te vrijednosti odbacivane su i tretirane kao kadrovi bez detektirane ruke, kako bi se spriječilo prenošenje pogrešnih klasifikacija gesta na robota.

Svih 21 ključnih točaka korišteno je u ovom radu u dvije svrhe: klasifikaciju gesta i procjenu položaja dlana. Za klasifikaciju gesta, četiri MCP ključne točke kostura dlana na indeksima 5, 9, 13 i 17 usrednjene su kako bi se definiralo središte dlana. Za svaki od četiri prsta, euklidska udaljenost od ključne točke vrha prsta do tog središta dlana uspoređivana je s udaljenošću od

odgovarajućeg PIP zgloba do istog središta. Ako je vrh prsta bio bliže dlanu nego PIP zglob za konfigurabilni faktor skale, prst je smatran savijenim. Ruka u kojoj su tri ili više prstiju istovremeno bili savijeni klasificirana je kao šaka, koja je služila kao ulaz sigurnosne sklopke mrtvog čovjeka, upravljačke naredbe brzine slane su robotu isključivo dok je operater održavao zatvorenu šaku.

Za procjenu položaja dlana, budući da MediaPipe Hands pruža samo normalizirane dvodimenzionalne koordinate slike i relativnu procjenu unutarnje dubine, a ne metričke trodimenzionalne položaje, za dobivanje stvarnih udaljenosti korišten je poravnati okvir dubine Intel RealSense D435 kamere. Koordinate piksela svake ključne točke korištene su za pronalazak odgovarajuće metričke vrijednosti dubine u okviru dubine, koja je zatim transformirana u trodimenzionalni prostor pomoću unutarnje matrice parametara kamere putem funkcije `rs2_deproject_pixel_to_point` iz RealSense SDK-a. Dobiveni trodimenzionalni položaj središta dlana, izražen u metrima u odnosu na koordinatni sustav kamere, objavljivao se kao tema u ROS 2 okruženju i koristio ga je čvor za daljinsko upravljanje za izračun proporcionalnih upravljačkih naredbi brzine za robota.

6. OPEN CV BIBLIOTEKA

OpenCV (Open Source Computer Vision Library) je sveobuhvatna biblioteka otvorenog koda namijenjena obradi slika i računalnom vidu u stvarnom vremenu, koja pruža nekoliko tisuća optimiziranih algoritama za zadatke poput filtriranja slike, detekcije rubova, kalibracije kamere, optičkog toka i prepoznavanja objekata. Biblioteka je izvorno napisana u programskom jeziku C++, no dostupna je i putem Python sučelja (cv2) koje je korišteno u ovom radu. Temeljni podatkovni model OpenCV-a oslanja se na NumPy polja: svaki okvir slike predstavljen je kao trodimenzionalno polje oblika (visina, širina, broj kanala), pri čemu su vrijednosti piksela pohranjene kao 8-bitni cijeli brojevi bez predznaka. Važna konvencija biblioteke je da boje interno pohranjuje redoslijedom plavog, zelenog i crvenog kanala (BGR), za razliku od uobičajenog RGB redoslijeda koji koriste druge biblioteke poput MediaPipea; zbog toga je u ovom radu svaki okvir boje prije prosljeđivanja modelu za detekciju ključnih točaka ruke pretvoren iz BGR u RGB prostor boja pozivom funkcije `cv2.cvtColor` s parametrom `cv2.COLOR_BGR2RGB`.

Okviri dubine koje pruža Intel RealSense D435 pohranjena su kao 16-bitna polja s vrijednostima izraženima u milimetrima, što ih čini neprikladnima za izravni vizualni prikaz. U ovom je radu okvir dubine za potrebe grafičkog sučelja obrađen u dva koraka: funkcijom `cv2.convertScaleAbs` vrijednosti su linearno skalirane i svedene na 8-bitni raspon (0-255), nakon čega je funkcijom `cv2.applyColorMap` primijenjena pseudokolorna mapa `cv2.COLORMAP_JET`, koja bliže udaljenosti prikazuje hladnim (plavim) bojama, a dalje udaljenosti toplim (crvenim) bojama. Na taj način vizualizirani okvir dubine prikazuje se uz okvir boje, dajući operateru intuitivan pregled prostorne geometrije scene u stvarnom vremenu.

Grafičko sučelje u ovom radu izgrađeno je isključivo korištenjem primitiva za crtanje OpenCV-a, bez upotrebe posebnog okvira za grafičko korisničko sučelje. Funkcija `cv2.rectangle` korištena je za crtanje obojenih pravokutnih traka kao pozadine za tekstualne informacije, čime su postignuti vizualno odvojeni informacijski paneli. Funkcija `cv2.putText` s fontom `cv2.FONT_HERSHEY_SIMPLEX` i zastavicom `cv2.LINE_AA` za glađenje rubova korištena je za prikazivanje podataka o prepoznatoj gesti, pouzdanosti detekcije, statusu sigurnosne sklopke, odmaku ruke od neutralnog položaja i statusu kalibracije. Isti tekst iscrtavan je dva puta — jednom crnom bojom s većom debljinom linije kako bi se dobila sjena, a zatim u željenom obojenju s manjom debljinom — čime je postignut efekt obruba koji tekst čini čitkim neovisno o podlozi. Funkcija `cv2.circle` korištena je za preslikavanje ključnih točaka ruke iz okvira boje na odgovarajuće položaje u koloriranom okviru dubine, čime se operateru vizualno potvrđuje da su detektirane točke prostorno konzistentne između oba prikaza. Konačno, dva vizualna panela — okvir boje s ucrtanim skeletom ruke i prikazanim informacijama te kolorirani

okvir dubine — horizontalno su spojena pomoću NumPy funkcije `np.hstack`, dok je ispod njih vertikalnom kompozicijom (`np.vstack`) dodan statusni panel koji prikazuje kalibracijski napredak i odmake od neutralnog položaja u oba smjera. Rezultirajući kompozitni okvir prikazuje se pozivom `cv2.imshow` unutar prozora kreiranog s `cv2.namedWindow`, a tipkovnički unosi operatera obrađuju se funkcijom `cv2.waitKey`, koja je u ovom radu konfigurirana s vremenskim ograničenjem od jedne milisekunde kako ne bi blokirala petlju kamere — pritisak tipke C pokreće postupak kalibracije neutralnog položaja, dok tipka Q ili tipka Escape pokreće uredno gašenje čvora.

7. SUSTAV ZA UPRAVLJANJE POMOĆU GESTI RUKE

Problem upravljanja mobilnim robotom pomoću gesti ruke rješava se razvojem dva ROS 2 čvora koji koriste MediaPipe Hands za detekciju ruke i klasifikaciju gesta, te Intel RealSense D435 Depth kameru za procjenu trodimenzionalnog položaja dlana.

Prije razvijanja i pisanja koda za čvorove, osmišljena je logika sustava, definirane su geste koje će se koristiti za upravljanje robotom, teme i poruke koje će se koristiti za komunikaciju između čvorova te napisan pseudokod koji opisuje način rada svakog čvora. Također, instalirani su svi potrebni ROS 2 paketi i biblioteke, uključujući MediaPipe i Intel RealSense SDK 2.0, te su testirani njihovi osnovni primjeri kako bi se osiguralo da su ispravno instalirani i funkcionalni.

Prvi razvijeni čvor, `hand_gesture_recognizer`, odgovoran je za prepoznavanje gesta ruke i objavljivanje rezultata u ROS 2 okruženju. Drugi čvor, `hand_gesture_controller`, pretvara prepoznate geste i procijenjeni položaj dlana u upravljačke naredbe brzine za robota, koje se objavljuju na standardnoj ROS 2 temi `geometry_msgs/Twist`.

Prije implementacije sustava na fizičkog mobilnog robota ASTRO, razvijeni su i testirani u simuliranom okruženju pomoću `rviz-a`, gdje su se provjeravale ispravnost prepoznavanja gesta, točnost procjene položaja dlana i odgovarajuće generiranje upravljačkih naredbi brzine. Nakon što je sustav uspješno testiran u simulaciji, implementiran je na fizički robot ASTRO, gdje je dodatno testiran u stvarnom vremenu s operaterom.

Zbog jednostavnosti, čvorovi će u ovom radu biti opisani pojedinačno i preko pseudokoda. Poveznica za cijeli paket i kodove svih čvorova nalazi se u Prilogu I.

7.1. Prepoznavanje geste ruke

Logika osmišljena za prepoznavanje gesta ruke temelji se na klasifikaciji ruke u dva stanja: otvorena i zatvorena šaka. Kao i većina robotskih sustava, i ovaj sustav koristi sigurnosnu sklopku mrtvog čovjeka (engl. *dead man's switch*), što znači da se poruke o položaju dlana, a time i brzine robota, šalju isključivo dok operater drži šaku zatvorenom. Ukoliko se šaka otvori, robot se odmah zaustavlja i ne prima nove upravljačke naredbe dok se šaka ponovno ne zatvori. Osim geste šake, sustav prati i udaljenost ruke od kalibriranog početnog položaja u horizontalnoj ravnini. Iz te udaljenosti se kasnije izračunava proporcionalna brzina kretanja robota, što je detaljnije opisano u poglavlju 6.2.

U ovom čvoru je korišten i OpenCV za obradu slike i vizualni prikaz povratnih informacija operateru preko grafičkog korisničkog sučelja. OpenCV je korišten za prikaz slike iz kamere, označavanje detektirane ruke i prikaz prepoznate geste. Također, korišten je i za prikaz poravnate

slike dubine te video prijenos s kamere pozicionirane s prednje strane robota kako bi operater imao bolju povratnu informaciju o poziciji robota u prostoru.

Uvezeni moduli u ovom čvoru su:

- rclpy
- rclpy.node
- geometry_msgs.msg.Point
- std_msgs.msg.String
- std_msgs.msg.Header
- sensor_msgs.msg.CompressedImage
- pyrealsense2
- mediapipe
- cv2
- numpy
- time

Nakon inicijalizacije čvora, definirane su sljedeće varijable i parametri:

```
1 self.PUBLISH_RATE           = 20
2 self.MIN_CONFIDENCE         = 0.6
3 self.DEBOUNCE_FRAMES        = 5
4 self.FIST_CURL_THRESHOLD     = 0.8
5 self.CALIBRATION_DURATION    = 3.0
6
7 self.confirmed_gesture      = 'OPEN'
8 self.pending_gesture        = 'OPEN'
9 self.pending_count           = 0
10 self.latest_position         = None
11 self.latest_confidence       = 0.0
12 self.last_known_position    = Point()
13
14 self.null_position           = None
15 self.is_calibrating          = False
16 self.calibration_start_time = 0.0
```

Jedni od važnijih parametara koji nisu striktno za kameru su MIN_CONFIDENCE, DEBOUNCE_FRAMES i FIST_CURL_THRESHOLD. MIN_CONFIDENCE definira minimalnu pouzdanost klasifikacije geste ruke kako bi se smatrala valjanom. To smanjuje broj pogrešnih klasifikacija koje se mogu pojaviti zbog lošeg osvjetljenja, preklapanja ruke s drugim objektima ili nepreciznosti u detekciji. Parametar je određen eksperimentalno, ukoliko je pouzdanost pre visoka, sustav nikada neće detektirati zatvorenu šaku. Isto tako, ukoliko je pouzdanost pre malena, sustav će lako krivo detektirati gestu koju operater ne pokazuje, što može dovesti do

nesigurnog ponašanja robota i trzaja. `DEBOUNCE_FRAMES` definira broj uzastopnih sličica koje moraju biti klasificirane kao šaka ili otvorena ruka prije nego li se promijeni stanje potvrđene geste. Taj parametar je važan jer MediaPipe Hands može ponekad pogrešno klasificirati ruku, a `DEBOUNCE_FRAMES` omogućuje da se pogreške filtriraju, što sprečava trzaje robota pri kretanju. `FIST_CURL_THRESHOLD` definira prag za klasifikaciju geste šake na temelju omjera udaljenosti vrha prsta i PIP zgloba od središta dlana.

Ostali parametri definirani u ovom čvoru su za kalibraciju početnog položaja ruke, praćenje trenutne pozicije i pohranu posljednje poznate pozicije ruke.

Nakon definiranja početnih parametara, kreirani su ROS 2 pretplatnici i objavljiivači.

```

1 Create Publisher: '/hand_pose' (HandPose message)
2 Create Publisher: '/null_position' (Point message)
3 Create Subscriber: '/camera/color/image_raw/compressed' -> robot_camera_callback()
4 Create Subscriber: '/obstacle_status' -> obstacle_status_callback()

```

HandPose poruka se objavljuje na temu `/hand_pose` i sadrži trodimenzionalnu poziciju ruke, prepoznatu gestu i ocjenu pouzdanosti. Kasnije se koristi za izračun upravljačkih naredbi brzine robota. Null_position poruka se objavljuje na temu `/null_position` i sadrži trodimenzionalnu poziciju ruke u trenutku kada je operater započeo kalibraciju. Pretplatnici se kreiraju na temu `/camera/color/image_raw/compressed`, koja sadrži komprimirane slike iz Intel RealSense D435 kamere, te na temu `/obstacle_status`, koja sadrži status prepreke i koristi se za zaustavljanje robota u slučaju da sustav za izbjegavanje prepreka detektira prepreku u blizini robota.

Dalje u kodu slijedi inicijalizacija MediaPipe Hands modela, intrinzika i poravnanja kamere te kreiranje tajmera.

```

1 Initialize RealSense pipeline (Color & Depth @ 640x480, 30fps)
2 Initialize MediaPipe Hands model (min_detection_confidence, max_num_hands=1)
3
4 Create Timer(1/30s) -> camera_loop()
5 Create Timer(1/20s) -> publish_latest_state()

```

Pri inicijalizacije MediaPipe Hands modela, postavljen je maksimalan broj ruku koji može biti detektiran u jednom trenutku. MediaPipe Hands model radi na principu da ukoliko detektira ruku, ona se prati dok ne napusti kadar ili dok pouzdanost klasifikacije ne padne ispod praga. Parametar je postavljen na 1 jer se u ovom radu koristi samo jedna ruka operatera te se na ovaj način sprječava pogrešno detektiranje druge ruke operatera ili ruke druge osobe u kadru.

Glavna funkcija programa, `camera_loop()`, poziva se svakih 1/30 sekundi i obrađuje sliku iz kamere.

```

1 function camera_loop():
2     color_frame, depth_frame = RealSense.get_aligned_frames()

```

```

3     IF frames are invalid:
4         Debounce_Gesture('OPEN')
5         RETURN

```

Prvi dio funkcije je akvizicija i poravnanje kadra. Funkcija `RealSense.get_aligned_frames()` dohvaća okvire boje i dubine iz Intel RealSense D435 kamere i poravnava ih tako da svaki piksel u okviru boje odgovara istom pikselu u okviru dubine. Ukoliko su okviri neispravni, funkcija `Debounce_Gesture` poziva se s argumentom 'OPEN' kako bi se osiguralo da robot prestane primati upravljačke naredbe i zatim se funkcija prekida i vraća. Također, potrebno je pretvoriti kadrove u numpy polja, napraviti konverziju iz BGR u RGB prostor boja i napraviti vizualizaciju okvira dubine za prikaz u grafičkom korisničkom sučelju.

Nakon toga, MediaPipe Hands model obrađuje okvir boje i vraća rezultate detekcije ruke i ključnih točaka.

```

1     results = MediaPipe.process(color_frame)
2     IF hand NOT detected OR confidence < MIN_CONFIDENCE:
3         Debounce_Gesture('OPEN')
4         Render_Dashboard_GUI()
5         RETURN

```

Ukoliko je pouzdanost detekcije ruke manja od praga `MIN_CONFIDENCE`, funkcija `Debounce_Gesture` poziva se s argumentom 'OPEN' kako bi se osiguralo da robot prestane primati upravljačke naredbe. Također, potrebno je pozvati funkciju `Render_Dashboard_GUI()` kako bi se prikazali vizualni podaci u grafičkom korisničkom sučelju.

```

1     landmarks_2d = results.multi_hand_landmarks[0]
2     raw_gesture = Classify_Gesture(landmarks_2d)
3     Debounce_Gesture(raw_gesture)

```

Sljedeći korak je dohvat ključnih točaka ruke i klasifikacija geste. Funkcija `Classify_Gesture()` koristi omjer udaljenosti vrha prsta i PIP zgloba od središta dlana kako bi odredila je li ruka otvorena ili zatvorena šaka.

```

1     function Classify_Gesture(landmarks_2d):
2         palm_center = Calculate_Mean(landmarks_2d[5, 9, 13, 17])
3         curled_fingers_count = 0
4
5         FOR EACH finger IN [index, middle, ring, pinky]:
6             tip_dist = Distance(finger.tip, palm_center)
7             pip_dist = Distance(finger.pip, palm_center)
8
9             IF tip_dist < pip_dist * FIST_CURL_THRESHOLD:
10                 curled_fingers_count += 1
11

```

```

12         IF curled_fingers_count >= 3:
13             RETURN 'FIST'
14     ELSE:
15         RETURN 'OPEN'

```

Šaka je definirana kao zatvorena ukoliko su istovremeno savijena tri ili više prstiju. Taj broj je određen eksperimentalno, kada je prag bio manji, sustav je pogrešno klasificirao ruku kao šaku te kada je prag bio veći, sustav je pogrešno klasificirao ruku kao otvorenu. Stroži prag bi bio bolji za sigurnost, no za to su potrebni bolji uvjeti okoline za kameru ili bolji senzori. Nadalje, ako je jedan ili manje prstiju savijeno, ruka se klasificira kao otvorena, a sve ostalo se tretira kao neodređeno. Neodređeno stanje na upravljanje utječe isto kao i otvorena ruka, tj. robot prestaje primati upravljačke naredbe. Nadalje, ponovno se poziva funkcija `Debounce_Gesture()` kako bi se osiguralo da se pogrešne klasifikacije filtriraju i da se robot ne trza pri kretanju.

```

1     landmarks_3d = Deproject_2D_to_3D(landmarks_2d, depth_frame, camera_intrinsics)
2     palm_center = Compute_Mean_3D_Position(landmarks_3d[knuckles])
3     state.position_3d = palm_center

```

Dalje, budući da MediaPipe Hands model pruža samo normalizirane dvodimenzionalne koordinate slike i relativnu procjenu unutarnje dubine, a ne metričke trodimenzionalne položaje, za dobivanje stvarnih udaljenosti koristi se poravnati okvir dubine Intel RealSense D435 kamere. Funkcija `Deproject_2D_to_3D()` koristi koordinate piksela svake ključne točke kako bi pronašla odgovarajuću metričku vrijednost dubine u okviru dubine, koja se zatim transformira u trodimenzionalni prostor pomoću unutarnje matrice parametara kamere putem funkcije `rs2_deproject_pixel_to_point` iz RealSense SDK-a. Nakon toga, izračunava se središte dlana kao srednja vrijednost 3D pozicija MCP zglobova kostura dlana, koje se nalaze na indeksima 5, 9, 13 i 17. Dobivena pozicija središta dlana pohranjuje se u varijablu `state.position_3d`.

```

1     IF calibration_is_active:
2         IF elapsed_time >= CALIBRATION_DURATION:
3             state.null_position = state.position_3d
4             Publish('/null_position', state.null_position)
5             Stop_Calibration()

```

Kako bi se iz pozicije ruke izračunale upravljačke naredbe brzine, potrebno je odrediti referentnu poziciju ruke, tj. neutralni položaj, od kojeg se računa udaljenost ruke u horizontalnoj ravnini. Funkcija za kalibraciju aktivira se pritiskom tipke C na tipkovnici, nakon čega se mjeri vrijeme određeno parametrom `CALIBRATION_DURATION`. Nakon što je proteklo dovoljno vremena, trenutna pozicija ruke se pohranjuje u varijablu `state.null_position` i objavljuje na temu `/null_position`. Nakon toga, kalibracija se zaustavlja i sustav je spreman za objavljivanje informacija o poziciji i gesti ruke.

```

1  Render_Dashboard_Grid_2x2(color_frame, depth_colormap, robot_image, obstacle_status)
2
3  key = Read_Keyboard_Input()
4  IF key == 'C': Start_Calibration_Timer()
5  IF key == 'Q': Shutdown_Node()

```

Kako bi sustav bio što intuitivniji i jednostavniji za korištenje, razvijeno je grafičko korisničko sučelje koje prikazuje četiri panela. Gornji lijevi panel prikazuje video prijenos s vanjske kamere s ucrtanim skeletom ruke i ključnim točkama te tekstualne informacije o prepoznatoj gesti, pouzdanosti detekcije i upute za operatera poput unosa s tipkovnice za kalibraciju i gašenje sustava. Gornji desni panel prikazuje kolorirani okvir dubine s ucrtanim ključnim točkama ruke, što operateru daje informaciju o prostornoj poziciji ruke u odnosu na okolinu. Uz kolorirani okvir dubine nalazi se i legenda koja prikazuje povezanost boja i udaljenosti. Donji lijevi panel prikazuje video prijenos s kamere pozicionirane s prednje strane robota, što operateru daje bolju povratnu informaciju o poziciji robota u prostoru. Također, služi operateru za nadođenje u situacijama kada se robot ne nalazi u operaterovom vidnom polju. U tom slučaju, taj video prijenos je jedini način da operater orijentira robota u točnom smjeru. Donji desni panel jest statusni panel koji prikazuje razne informacije o stanju sustava poput statusa kalibracije i statusa sustava za detekciju prepreka. U ovom panelu operater dobiva informaciju kada mu se oduzima mogućnost upravljanja robotom u slučaju nailaska na prepreku, u kojem je stadiju algoritam za zaobilazanje prepreke te, naposljetku, kada mu je vraćena kontrola nad robotom.

Naposljetku, definirana je funkcija koja objavljuje najnovije stanje sustava.

```

1  function publish_latest_state():
2      msg = HandPose()
3      msg.timestamp = Current_Time()
4      msg.gesture    = state.confirmed_gesture
5      msg.confidence = state.latest_confidence
6      msg.position   = state.position_3d IF valid ELSE state.last_known_position
7
8      Publish('/hand_pose', msg)

```

Ovaj čvor prikuplja, obrađuje i objavljuje sve potrebne informacije za računanje brzina robota i upravljanje mobilnim robotom ASTRO pomoću pozicije i geste ruke.

7.2. Izračun brzina robota

Jednom kada je određena udaljenost šake od referentnog položaja u planarnoj ravnini i je li šaka otvorena ili zatvorena, iz tih podataka može se izračunati brzina robota, linearna i kutna, proporcionalna određenoj udaljenosti. Sve koordinate pozicije šake koje se koriste u ovom čvoru izražene su u koordinatnom sustavu optičkog okvira kamere (engl. *camera optical frame*), koji

je standardna konvencija za RGB-D kamere. U tom koordinatnom sustavu os x pokazuje udesno, os y prema dolje, a os z u dubinu scene, tj. od kamere prema promatranoj sceni. Iz toga slijedi da pomak šake u smjeru osi x odgovara lateralnom pomaku (lijevo-desno), koji se preslikava na kutnu brzinu robota, dok pomak u smjeru osi z odgovara promjeni udaljenosti šake od kamere, koji se preslikava na linearnu brzinu robota.

Uvezeni moduli u ovom čvoru su:

- rclpy
- rclpy.node
- geometry_msgs.msg.Twist
- geometry_msgs.msg.Point
- hand_teleop_msgs.msg.HandPose
- math

Modul math koristi se za matematičke funkcije korištene kod računanja brzine.

Nakon inicijalizacije čvora, potrebno je odrediti početne parametre. Svi parametri su određeni eksperimentalno.

```

1  class HandTeleopNode(ROS2_Node):
2
3      function __init__():
4          Initialize ROS 2 node "hand_teleop_node"
5
6          Declare and Load Parameters:
7              - deadzone_radius, max_linear_speed, max_angular_speed
8              - map_range_linear, map_range_angular
9              - smoothing_alpha (EMA filter coefficient)
10             - tracking_timeout, publish_rate (20 Hz)
11
12         state = {
13             'neutral_position': NULL,
14             'latest_hand_pos': NULL,
15             'latest_gesture': "OPEN",
16             'last_msg_time': NULL,
17             'filtered_linear_x': 0.0,
18             'filtered_angular_z': 0.0
19         }
```

Parametar `deadzone_radius` potreban je kako bi se mali pomaci šake oko kalibrirane točke ignorirali. To je važno jer se ovaj način upravljanja bazira na gruboj motorici operatera te moramo filtrirati i ignorirati male drhtaje i pokrete ruku i tijela kako se robot ne bi trzao. Parametri `map_range_linear` i `map_range_angular` definiraju fizikalni raspon kretanja šake koji odgovara punoj brzini robota. Konkretno, `map_range_linear` predstavlja udaljenost u metrima kojom operater mora pomaknuti šaku prema ili od kamere u odnosu na referentni položaj kako bi

robot dostigao svoju maksimalnu linearnu brzinu. Primjerice, postavljanjem vrijednosti na 0,20 m postiže se da pomak šake od 20 cm od referentne točke prema kameri rezultira maksimalnom brzinom kretanja robota naprijed. Isti princip vrijedi za `map_range_angular` u lateralnom smjeru. Manji raspon daje osjetljivije, brže upravljanje, dok veći raspon daje precizniju, finiju kontrolu pri manjim pomacima. Smanjenju trzaja robota pridonosi i `smoothing_alpha` parametar koji se koristi u smoothing filteru. Parametri za brzinu koriste se za skaliranje brzine u odnosu na udaljenost. `Publish_rate` odgovara onome definiranom u prethodnom čvoru.

Također su definirana i početna stanja tako da je pri pokretanju sustava sve u mirovanju radi sigurnosnih razloga.

```

1      Create Subscriber: '/hand_pose'      -> on_hand_pose_received()
2      Create Subscriber: '/null_position' -> on_null_position()
3      Create Publisher:  '/cmd_vel_hand' (Twist message)
4
5      Create Timer(1 / publish_rate) -> publish_velocity()

```

Potom je potrebno stvoriti komunikaciju s ostalim čvorovima kako bi se dobile potrebne informacije. Čvor se pretplaćuje na `/hand_pose` i `/null_position` za informacije o gesti ruke i udaljenosti od referentnog položaja, isto kao i informaciju o referentnom položaju. Ovaj čvor objavljuje poruku oblika Twist na temu `/cmd_vel_hand`, na koju se pretplaćuje paket `twist_mux`. Taj paket u svakom trenutku prati više konkurentnih izvora naredbi brzine poput upravljanje upravljačkom palicom, upravljanje gestama ruke i izbjegavanje prepreka. Paket prosljeđuje naredbu izvora s najvišim konfiguriranim prioritetom na temu `/cmd_vel`, koju čita upravljač robota ASTRO. Na taj način, ukoliko operater istovremeno koristi upravljačku palicu i upravljanje gestama, uprednost ima način upravljanja s većim prioritetom. Uz pretplatnike i objavljivače, stvoren je i tajmer kako bi se čvor mogao sinkronizirati s `hand_capture` čvorom.

```

1      function on_null_position(msg):
2          state.neutral_position = msg.position
3
4
5      function on_hand_pose_received(msg):
6          state.last_msg_time     = Current_Time()
7          state.latest_hand_pos   = msg.position
8          state.latest_gesture    = msg.gesture

```

Funkcije `on_null_position` i `on_hand_pose_recieved` služe da dohvaćanje informacija dobivenih od tema na koje je čvor pretplaćen kako bi se mogle koristiti u daljnjem kodu.

Potom se pokreće glavna funkcija programa, `publish_velocity`, koja služi za objavu brzina motora robota. Funkcija se sastoji od sedam dijelova.

```

1      function publish_velocity():

```

```

2         IF state.neutral_position is NULL OR state.last_msg_time is NULL:
3             Publish_Zero_Twist()
4             RETURN
5
6         IF (Current_Time() - state.last_msg_time) > tracking_timeout:
7             Publish_Zero_Twist()
8             RETURN
9
10        IF state.latest_gesture != "FIST":
11            Publish_Zero_Twist()
12            RETURN

```

Prvi dio funkcije ima sigurnosnu ulogu. Razmatraju se tri uvjeta pri kojima se umjesto stvarne brzine objavljuje nulta brzina. Prvi uvjet provjerava je li kalibracija referentnog položaja uopće provedena te je li pristigla barem jedna poruka s pozicijom šake. Ukoliko nije, robot miruje. Drugi uvjet jest vremensko ograničenje praćenja (`tracking_timeout`), koje se aktivira kada od posljednje primljene poruke s pozicijom šake protekne više vremena nego što je dozvoljeno. Do toga može doći ako šaka napusti vidno polje kamere, ako MediaPipe izgubi praćenje ruke uslijed lošeg osvjetljenja ili brzog pokreta, ili ako dođe do prekida prijenosa podataka između čvorova. Vrijednost timeouta namjerno je postavljena malo ispod timeouta koji koristi `twist_mux` (0,5 s) kako bi se osiguralo da ovaj čvor sam zaustavi robota i objavi nultu brzinu prije nego što `twist_mux` samostalno reagira na nedostatak poruka. Treći uvjet jest sigurnosna sklopka mrtvog čovjeka. Robot prima naredbe za kretanje isključivo dok operater drži zatvorenu šaku. Otvaranje šake, prijelazno stanje geste ili gubitak detekcije rezultiraju trenutnom objavom nulte brzine.

```

1         dx = state.latest_hand_pos.x - state.neutral_position.x
2         dz = state.neutral_position.z - state.latest_hand_pos.z

```

Sljedeći dio funkcije služi za računanje pomaka, udaljenosti, od kalibrirane neutralne pozicije. Važno je napomenuti zašto se pomak u dubinu `dz` računa kao razlika u obrnutom redoslijedu u odnosu na `dx`. Kamera Intel RealSense D435 mjeri dubinu kao udaljenost od objektiva kamere, što znači da se vrijednost koordinate `z` povećava što je ruka dalje od kamere. Kako bi se postiglo intuitivno upravljanje, tj. pomicanje šake prema kameri treba rezultirati kretanjem robota naprijed, potrebno je preokrenuti predznak. Oduzimanjem trenutne pozicije šake od referentne vrijednosti (`neutral_position.z - latest_hand_pos.z`) dobiva se pozitivan `dz` kada se šaka pomiče prema kameri, što se zatim preslikava na pozitivnu linearnu brzinu, odnosno kretanje robota prema naprijed. U suprotnom, pomicanje šake od kamere daje negativan `dz` i robot se kreće unatrag.

```

1         IF Euclidean_Distance(dx, dz) < deadzone_radius:
2             dx = 0.0
3             dz = 0.0

```

Prije računanja brzine iz pomaka definiranog u prijašnjem koraku, potrebno je filtrirati pomake manje od radijusa mrtve zone koja je definirana u početnim parametrima. Ukoliko je pomak manji od zadanog radijusa, pomak će se smatrati da je jednak nuli.

```

1      norm_forward = Clamp(dz / map_range_linear, -1.0, 1.0)
2      norm_turn    = Clamp(dx / map_range_angular, -1.0, 1.0)

```

Potom je potrebno provesti normalizaciju i ograničavanje kojima se fizikalni pomak ruke u metrima pretvara u sigurnu upravljački signal od -1.0 do 1.0. Normalizacija pretvara izmjerenu fizikalnu udaljenost ruke od nulte točke u relativni postotak u odnosu na zadani maksimalni raspon pokreta definiran početnim parametrima. Ograničavanje osigurava da dobivena normalizirana vrijednost strogo matematičkom granicom ne može izaći izvan raspona [-1.0, 1.0], bez obzira koliko daleko operater pomakne ruku izvan predviđenog radnog prostora. Svrha ove operacije je sigurnost i stabilnost robota, linearizacije pruža operateru proporcionalan osjećaj vožnje a sigurnosni strop sprječava da robot zbog prevelikog pokreta ruke dobije naredbu za brzinu koja premašuje maksimalne fizičke granice robota.

```

1      target_linear_x  = norm_forward * max_linear_speed
2      target_angular_z = norm_turn * max_angular_speed

```

Peti korak jest skaliranje normaliziranih vrijednosti na stvarne fizikalne brzine množenjem s parametrima `max_linear_speed` i `max_angular_speed`. Ove granične vrijednosti odgovaraju hardverskim ograničenjima robota ASTRO definiranim u upravljaču `astro_hardware_interface`. Maksimalna linearna brzina iznosi $\pm 1,0$ m/s, a maksimalna kutna brzina $\pm 1,0$ rad/s. Budući da je ulazna normalizirana vrijednost ograničena na raspon [-1.0, 1.0] u prethodnom koraku, umnožak s ovim graničnim vrijednostima jamči da izlazna naredba brzine nikada neće premašiti fizikalne granice robota, neovisno o položaju ruke operatera.

```

1      state.filtered_linear_x = (smoothing_alpha * target_linear_x) + ((1 -
    ↪ smoothing_alpha) * state.filtered_linear_x)
2      state.filtered_angular_z = (smoothing_alpha * target_angular_z) + ((1 -
    ↪ smoothing_alpha) * state.filtered_angular_z)

```

Nakon skaliranja primjenjuje se eksponencijalno klizno usrednjavanje (EMA, engl. *Exponential Moving Average*), opisano formulom:

$$y_k = \alpha \cdot x_k + (1 - \alpha) \cdot y_{k-1} \quad (1)$$

gdje je x_k trenutno izračunata ciljna brzina, y_{k-1} prethodno filtrirana brzina, a α koeficijent izgladivanja u rasponu (0,1]. Vrijednost α bliža nuli daje sporiji, gladi odziv filtera jer se više uzima u obzir prethodna vrijednost, dok vrijednost bliža jedan daje brži odziv, ali manje izgladen signal. Svrha ovog filtera jest smanjenje naglih skokova brzine koji bi mogli nastati zbog šuma u

mjerenjima kamere ili brzih nevoljnih pokreta ruke operatera, čime se postiže glađe i sigurnije kretanje robota.

```

1      msg = Twist()
2      msg.linear.x = state.filtered_linear_x
3      msg.angular.z = state.filtered_angular_z
4      Publish('/cmd_vel_hand', msg)

```

Završni korak funkcije jest objavljivanje brzina izračunatih ovom funkcijom kako bi ju robot mogao iskoristiti.

```

1  function Publish_Zero_Twist():
2      state.filtered_linear_x = 0.0
3      state.filtered_angular_z = 0.0
4
5      msg = Twist()
6      msg.linear.x = 0.0
7      msg.angular.z = 0.0
8      Publish('/cmd_vel_hand', msg)

```

Naposlijetku, potrebno je vratiti memoriju filtera u početno stanje kako bi se prevenirao iznenadni skok akceleracije pri ponovnom pokretanju.

Opisanim čvorom zaokružena je izrada cjelokupnog lanca obrade podataka koji omogućuje upravljanje robotom ASTRO pokretima ruke, od snimanja geste kamerom, preko klasifikacije i kalibracije, do izračuna i objave proporcionalnih naredbi brzine. Svi opisani čvorovi zajedno tvore funkcionalan sustav koji je spreman za integraciju u postojeću arhitekturu robota putem paketa `twist_mux`.

Radi lakšeg pokretanja sustava pomoću samo jedne naredbe, stvoren je paket koji sadrži dva čvora opisana u ovom poglavlju te datoteka za pokretanje.

```

1  def generate_launch_description():
2
3      twist_mux_config = os.path.join(
4          get_package_share_directory('astro'),
5          'config',
6          'twist_mux.yaml'
7      )
8
9      return LaunchDescription([
10
11          Node(
12              package='hand_teleop',
13              executable='hand_capture_node',
14              name='hand_capture_node',
15              output='screen',

```

```

16     ),
17
18     Node(
19         package='hand_teleop',
20         executable='hand_teleop_node',
21         name='hand_teleop_node',
22         output='screen',
23         parameters=[{
24             'deadzone_radius' : 0.03,
25             'max_linear_speed' : 1.0,
26             'max_angular_speed' : 1.0,
27             'map_range_linear' : 0.20,
28             'map_range_angular' : 0.20,
29             'smoothing_alpha' : 0.3,
30             'tracking_timeout' : 0.4,
31             'publish_rate' : 20.0,
32         }]
33     ),
34
35     Node(
36         package='twist_mux',
37         executable='twist_mux',
38         name='twist_mux',
39         output='screen',
40         parameters=[twist_mux_config],
41         remappings=[('/cmd_vel_out', '/cmd_vel')],
42     ),
43 ])

```

Pokretanjem ove datoteke pokreću se sljedeći čvorovi:

- hand_capture_node
- hand_teleop_node
- twist_mux.

7.3. Simulacija

Međutim, prije testiranja na stvarnom robotu, potrebno je provjeriti ispravnost implementiranog sustava u kontroliranim uvjetima kako bi se otklonili potencijalni problemi bez rizika od oštećenja opreme ili ugrožavanja sigurnosti. U tu svrhu provedena je provjera funkcionalnosti sustava u simulacijskom okruženju RViz, alatom za vizualizaciju tema i stanja robota unutar ROS 2 ekosustava. Budući da RViz za pravilno funkcioniranje zahtijeva prisutnost odometrijskih podataka koje stvarni robot generira putem enkodera, a koji nisu dostupni u simulacijskom okruženju bez pokretanja cjelokupnog hardverskog sloja, bilo je potrebno implementirati dodatni čvor koji generira sintetičke odometrijske podatke.

```

1  class DummyOdomNode(ROS2_Node):
2
3      function __init__():
4          Initialize ROS 2 node "dummy_odom_node"
5
6          state = {
7              'x': 0.0, 'y': 0.0, 'th': 0.0,
8              'vx': 0.0, 'wz': 0.0
9          }
10
11         last_time = Current_Time()
12
13         Create Subscriber: '/cmd_vel_hand' (Twist message) -> cmd_vel_callback()
14         Initialize TransformBroadcaster (tf2_ros)
15
16         Create Timer(0.05s) -> update_and_publish()

```

Početak programa inicijalizira sam čvor, određuje početne parametre, postavlja pretplatnika za dobivanje informacije o brzini, odašiljač transformacija i tajmer. Pozicija je zapisana u varijablama x i y, dok je orijentacija pohranjena u varijabli th. Linearna brzina je označena varijablom vx, a kutna varijablom wz.

```

1  function cmd_vel_callback(msg):
2      state.vx = msg.linear.x
3      state.wz = msg.angular.z

```

Stvoren je pretplatnički povratni poziv koji se pokreće automatski svaki put kada čvor upravljanja pošalje novu brzinu na temu /cmd_vel_hand. Funkcija prihvata poruku msg, izvuče brzine vx i wz te ih sprema u memoriju.

```

1  function update_and_publish():
2      current_time = Current_Time()
3      dt = current_time - last_time
4      last_time = current_time
5
6      delta_x = state.vx * Cosine(state.th) * dt
7      delta_y = state.vx * Sine(state.th) * dt
8      delta_th = state.wz * dt
9
10     state.x += delta_x
11     state.y += delta_y
12     state.th += delta_th

```

U glavnoj funkciji, update_and_publish, prvo se računa proteklo vrijeme. Zatim se provodi kinematska integracija te se preko diferencijalnog pogonskog kinematičkog modela računaju

brzine. Kinematska integracija koristi diferencijalnu jednadžbu gibanja diferencijalnog pogona. Na temelju trenutne orijentacije robota θ i izmjerene linearne brzine v_x te kutne brzine ω_z , pomaci po osima izračunavaju se kao:

$$\Delta x = v_x \cdot \cos(\theta) \cdot \Delta t \quad \Delta y = v_x \cdot \sin(\theta) \cdot \Delta t \quad \Delta \theta = \omega_z \cdot \Delta t \quad (2)$$

gdje je Δt proteklo vrijeme od prethodnog poziva funkcije. Integriranjem ovih diferencijalnih pomaka u svakom vremenskom koraku dobiva se procjena trenutne pozicije i orijentacije robota u ravnini, što je standardni postupak estimacije odometrije za mobilne robote s diferencijalnim pogonom.

```

1      transform = TransformStamped()
2      transform.header.stamp      = current_time
3      transform.header.frame_id = 'odom'
4      transform.child_frame_id  = 'base_footprint'
5
6      transform.translation = (state.x, state.y, 0.0)
7
8      transform.rotation.x = 0.0
9      transform.rotation.y = 0.0
10     transform.rotation.z = Sine(state.th / 2.0)
11     transform.rotation.w = Cosine(state.th / 2.0)

```

Sljedeći korak jest konstrukcija TF okvira transformacije. Koordinatni okvir `odom` predstavlja fiksnu referentnu točku u prostoru iz koje robot počinje kretanje i u kojoj se nalazi ishodište odometrije. Okvir `base_footprint` je koordinatni okvir pričvršćen za projekciju baze robota na tlo i pomiče se zajedno s robotom. Transformacija između ta dva okvira opisuje koliko se i u kojem smjeru robot pomaknuo od početne pozicije. Potom slijedi pretvorba kuta zakretanja u kvaternion. Rotacija se u ROS 2 ne izražava Eulerovim kutovima već kvaternionima, četvero-komponentnim matematičkim objektima koji jednoznačno opisuju orijentaciju u prostoru bez problema singularnosti karakterističnih za Eulerove kutove. Budući da se robot kreće samo u horizontalnoj ravnini, jedina relevantna rotacija je zakret oko osi z (kut θ), pa se kvaternion pojednostavljuje na oblik $(0, 0, \sin(\theta/2), \cos(\theta/2))$.

```

1      TF_Broadcaster.sendTransform(transform)

```

Naposlijetku se odašilju koordinate transformacije ROS 2 TF stabla. ROS 2 TF stablo (engl. *transform tree*) sustav je za praćenje prostornih odnosa između koordinatnih okvira robota u realnom vremenu. Svaki dio robota (baza, senzori, kotači) ima vlastiti koordinatni okvir, a TF stablo bilježi kako su ti okviri međusobno pozicionirani i orijentirani u svakom trenutku. RViz koristi TF stablo kako bi znao gdje se u prostoru nalaze pojedini dijelovi robota i kako ih

vizualizirati. Bez odometrijske transformacije između okvira `odom` i `base_footprint`, RViz ne može prikazati kretanje robota.

Na vanjsko računalo, na koje je bila spojena kamera Intel RealSense D435 i s kojeg se pokretao cijeli sustav, kloniran je repozitorij ASTRO s GitHub stranice CRTA laboratorija. Repozitorij sadrži sve potrebne datoteke za upravljanje robotom, uključujući i konfiguraciju za vizualizaciju u RViz okruženju s URDF modelom robota. Istovremeno su pokrenuti čvor `dummy_odom`, datoteka `hand_teleop.launch.py` koja pokreće čvorove za snimanje gesta i izračun brzina te `twist_mux`, a u RViz okruženju učitani su modeli robota ASTRO. Ispravnost sustava potvrđena je vizualnim praćenjem modela robota u RViz prozoru. Model se kretao naprijed, nazad i zakretao u skladu s pokretima šake operatera, s vidljivom reakcijom na otvaranje šake (zaustavljanje) i zatvaranje šake (nastavak kretanja), čime je potvrđena funkcionalna ispravnost implementiranog lanca upravljanja prije testiranja na stvarnom robotu.

(Upute za pokretanje simulacije nalaze se na GitHub repozitoriju CRTA laboratorija. Uz RViz su pokrenute `dummy_odom.py` i `hand_teleop.launch.py` datoteke.) fusnota

7.4. Integracija s mobilnim robotom ASTRO

Budući da je simulacijom potvrđena ispravnost sustava, završni korak je bio integrirati sustav s fizičkim mobilnim robotom ASTRO. U ovom radu korišten je ASTRO 3 te će se u buduću koristiti taj identifikacijski (ID) broj, no ovaj sustav je primjenjiv na bilo koji mobilni robot ASTRO ukoliko se ID broj zamijeni valjanom.

U prethodnim poglavljima opisano je kako je kloniran ASTRO repozitorij te kako je izrađena datoteka za pokretanje. Stoga, jedini preostali korak jest pokrenuti ASTRO robota i bežično ga spojiti s vanjskim računalom.

ASTRO se pokreće pritiskom tipki Power i JETSON. Tada je potrebno pričekati nekoliko minuta dok se ne pojavi Wi-Fi mreža pod nazivom Astro 3. Jednom kada je moguće, računalo se spaja na tu mrežu te je sklopovlje spremno za pokretanje sustava.

Fizički robot ASTRO 3 ponašao se u skladu s očekivanjima potvrđenim simulacijom. Zatvaranjem dlana u šaku deaktivirala se sklopka mrtvog čovjeka te je robot počinjao primati naredbe za kretanje. Pomicanjem šake prema kameri robot se kretao naprijed, pomicanjem od kamere unatrag, a lateralnim pomicanjem šake ulijevo ili udesno robot se zakretao u odgovarajućem smjeru. Brzina kretanja bila je proporcionalna odmaku šake od referentnog položaja, a otvaranjem šake robot se trenutno zaustavljao, čime je potvrđena ispravna funkcionalnost sigurnosne sklopke mrtvog čovjeka u uvjetima stvarnog upravljanja.

Optimalni parametri sustava određeni su eksperimentalno iterativnim postupkom testiranja na fizičkom robotu. Parametri koji su zahtijevali fino ugađanje uključuju `min_confidence`, `debounce_frames`, `fist_curl_threshold`, `deadzone_radius`, `smoothing_alpha` te `map_range_linear` i `map_range_angular`. Konačne vrijednosti svih parametara rezultat su

kompromisa između intuitivnosti upravljanja, stabilnosti kretanja i glatkoće vožnje robota.

Uspješnom integracijom i testiranjem na fizičkom robotu potvrđena je funkcionalna ispravnost cjelokupnog sustava upravljanja gestama ruke razvijenog u ovom radu. Sustav omogućuje intuitivno, bežično upravljanje mobilnim robotom ASTRO isključivo pokretima šake operatera, bez potrebe za fizičkim kontrolerom. Arhitektura temeljena na ROS 2 čvorovima i `twist_mux` paketu osigurava modularnost i jednostavnu proširivost. Novi načini upravljanja ili novi senzori mogu se dodati bez izmjena postojećeg koda.

zaključak (Razvijena rješenja, uključujući čvorove za snimanje i klasifikaciju gesta te izračun brzina i izbjegavanje prepreka, predstavljaju osnovu na kojoj se mogu graditi naprednije metode interakcije čovjeka i robota u budućim istraživanjima CRTA laboratorija.) zaključak

8. SUSTAV ZA ZAOBILAŽENJE PREPREKA

Objašnjen problem i rješenje.

9. ZAKLJUČAK

Tekst zaključka.

LITERATURA

- [1] Kraut, B.: Strojarski priručnik, Tehnička knjiga Zagreb, 1970.

PRILOZI

I. link za git